WEEK 5 – DATA SCIENCE AND VISUALIZATION

Dataset Cleaning and Preparation

Project	Data Analysis Project	**Dataset**	DataCoSupplyChainDataset.csv.xls
Syllabus Mapping	Week 5 – Data Cleaning	**Tool**	Google Colab

1. Introduction

Week 5 focuses on cleaning and preparing the DataCo supply-chain dataset before analysis. The workflow loads the file into Pandas, inspects its structure, checks duplicate records and missing values, standardizes text, converts date columns, removes selected unnecessary or sensitive columns, examines possible outliers, and exports a cleaned CSV file.

The notebook was prepared and run in Google Colab using Python. Final counts should be copied from the executed notebook output before submission.

2. Objectives

• Load the source dataset and inspect its dimensions, column names, data types and summary statistics.

• Identify duplicate rows and remove exact duplicates if present.

• Examine missing values and handle them using suitable rules.

• Remove fully empty columns and the mostly missing Order Zipcode field, as implemented in the notebook.

• Standardize whitespace in column names and text values.

• Convert order and shipping date fields to datetime format.

• Remove selected customer-identifying columns from the analysis copy.

• Inspect potential numerical outliers without automatically deleting them.

• Validate the cleaned data and save it as DataCoSupplyChain_Cleaned.csv.

3. Dataset Used

Dataset: DataCoSupplyChainDataset.csv.xls. The notebook reads the uploaded file with Pandas using the Latin-1 encoding and low-memory mode disabled. The original source file should be preserved separately.

The exact initial and final dimensions depend on the file uploaded and the cells executed. Record the actual values printed by df.shape in the Colab notebook rather than estimating them.

4. Tools and Technologies

Python, Google Colab, Pandas, NumPy and Matplotlib. Pandas is used for loading, inspecting, cleaning and exporting tabular data; NumPy supports numerical handling; Matplotlib is used for a missing-value chart when missing values remain.

5. Load Dataset and Inspect

The dataset is uploaded through the Google Colab file-upload dialog and loaded into a DataFrame named df.

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np

filename = next(iter(uploaded))
df = pd.read_csv(filename, low_memory=False, encoding="latin1")

print("Dataset loaded successfully!")
```

```
print("Rows and columns:", df.shape)
display(df.head())
df.info()
print(df.isnull().sum().sort_values(ascending=False).head(15))
display(df.describe())
```

6. Duplicate Check

Exact duplicate rows are counted and removed using drop_duplicates(). The notebook prints the DataFrame shape after this operation.

```
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

7. Missing Values and Empty Columns

The notebook removes columns that are entirely empty and removes Order Zipcode because it has a high proportion of missing values. Remaining text fields are filled with “Unknown” and remaining numerical fields with their median where applicable. Customer Zipcode is handled separately as a text-like identifier and missing values are filled with “Unknown”.

The executed notebook showed “Missing values: 0” in the displayed validation output. Re-run the final missing-value check after all cleaning cells to confirm the final state.

```
df = df.dropna(axis=1, how="all")
if "Order Zipcode" in df.columns:
    df = df.drop(columns=["Order Zipcode"])

if "Customer Zipcode" in df.columns:
    df["Customer Zipcode"] = (
        df["Customer Zipcode"].astype("string")
        .replace("nan", pd.NA).fillna("Unknown")
    )

for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna("Unknown")

for col in df.select_dtypes(include=np.number).columns:
    if df[col].isnull().any():
        df[col] = df[col].fillna(df[col].median())

print("Missing values:", df.isnull().sum().sum())
```

8. Text Standardization

Whitespace is removed from column names and from values in object-type columns. This reduces accidental category differences caused by leading or trailing spaces.

```
df.columns = df.columns.str.strip()
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()
```

9. Date Conversion

The order date and shipping date fields are converted to datetime. The errors='coerce' option converts unparseable values to missing datetime values, so the final missing-value check should be run after conversion.

```
date_columns = [
    "order date (DateOrders)",
    "shipping date (DateOrders)"
]
for col in date_columns:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors="coerce")
```

10. Remove Selected Customer Details

The notebook drops Customer Email, Customer Password, Customer Fname, Customer Lname and Customer Street when these columns exist. This reduces unnecessary personal information in the analysis dataset. Keep the original file private and do not publish sensitive data.

```
private_columns = [
    "Customer Email", "Customer Password",
    "Customer Fname", "Customer Lname", "Customer Street"
```

```
]
df = df.drop(
    columns=[col for col in private_columns if col in df.columns]
)
```

11. Inspect Data Types, Categories and Outliers

The notebook reviews data types and descriptive statistics, and prints frequent values for Type, Delivery Status, Shipping Mode and Order Status when those columns exist.

Potential outliers are identified with the Interquartile Range (IQR) rule for Sales, Order Item Quantity, Order Item Profit Ratio and Days for shipping (real). These are flagged for investigation only; they are not automatically removed because extreme values may be genuine business records.

```
for col in ["Sales", "Order Item Quantity",
            "Order Item Profit Ratio", "Days for shipping (real)"]:
    if col in df.columns:
        q1 = df[col].quantile(0.25)
        q3 = df[col].quantile(0.75)
        iqr = q3 - q1
        lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
        count = ((df[col] < lower) | (df[col] > upper)).sum()
        print(f"{col}: {count} potential outliers")
```

12. Final Validation and Export

The final validation checks the row and column count, total missing values and duplicate rows. The cleaned DataFrame is exported as a CSV file without adding the DataFrame index.

```
print("Final dataset shape:", df.shape)
print("Total missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())

df.to_csv("DataCoSupplyChain_Cleaned.csv", index=False)

from google.colab import files
files.download("DataCoSupplyChain_Cleaned.csv")
```

13. Results to Record

Fill the table using the actual output from the final executed notebook. The report deliberately does not invent dataset dimensions or duplicate counts.

Item	Record from Colab output
Initial dataset dimensions	Copy the shape printed immediately after loading the source file.
Duplicate rows removed	Record the duplicate count before drop_duplicates().
Columns removed	List columns actually removed (all-empty, Order Zipcode if present, and selected customer details if present).
Final dataset dimensions	Copy the final df.shape output.
Missing values after cleaning	The displayed notebook check reported 0; re-check after date conversion and all cleaning steps.
Duplicate rows after cleaning	Copy the final df.duplicated().sum() output.
Exported file	DataCoSupplyChain_Cleaned.csv

14. Conclusion

Week 5 establishes a cleaned version of the DataCo supply-chain dataset for subsequent analysis. The workflow covers data inspection, duplicate checking, missing-value treatment, text standardization, date conversion, removal of selected unnecessary personal details, outlier inspection and CSV export. The cleaned file should be validated using the final notebook output before it is submitted or used for later analysis.

15. Project Status

Week 5 – Dataset Cleaning: notebook cells and output-export steps are prepared. Before submission, verify that the final validation cell ran successfully, review the actual row and column counts, and retain both the Colab notebook and exported CSV.